Downloaded from: justpaste.it/ir9vj

1.Gauss elimination

```
def getMatrix(size):
    matrix = [[0] * size for _ in range(size)]
    for i in range(size):
        for j in range(size):
            matrix[i][j] = float(input(f"Enter element [{i}][{j}]: "))
    return matrix
def GaussElimination(n, A, b):
    for k in range(n - 1):
        for i in range(k + 1, n):
            factor = A[i][k] / A[k][k]
            for j in range(k, n):
                A[i][j] = A[i][j] - factor * A[k][j]
            b[i] = b[i] - factor * b[k]
    x = [0] * n
    x[n - 1] = b[n - 1] / A[n - 1][n - 1]
    for i in range(n - 2, -1, -1):
        total = b[i]
        for j in range(i + 1, n):
            total -= A[i][j] * x[j]
        x[i] = total / A[i][i]
    return x
N = int(input("Enter the size of the matrix: "))
print("Enter the matrix A: ")
matrix = []
for i in range(N):
    row = list(map(float,input().split()))
    matrix.append(row)
print("Enter the answer vector b: ")
b = list(map(float, input().split()))
ans = GaussElimination(N, matrix, b)
for _ in ans:
    print(_)
```

OUTPUT

```
Enter the size of the matrix: 3
Enter the matrix A:
2 1 -1
-3 -1 2
-2 1 2
Enter the answer vector b:
8 -11 -3
```

2.GAUSS ELI AND ROW PIVOTING

```
from fractions import Fraction
def printMatrix(A):
    for row in A:
        print("[",end="")
        for elem in row:
            print(round(Fraction(elem),5)," ",end="")
        print("]",end="")
        print()
def gauss_elimination(n, A, b):
    TOL = 1e-10
    for k in range(n - 1):

        # Check for zero pivot
        if abs(A[k][k]) <= TOL:
            print("The first row first column in 0")
            # Find the row with the largest pivot element
            max_row = k
            max_value = abs(A[k][k])

            for p in range(k + 1, n):
                if abs(A[p][k]) > max_value:
                    max_value = abs(A[p][k])
                    max_row = p

            # If no suitable pivot exists
            if max_value <= TOL:
                raise ValueError("No unique solution exists.")
```

```
            # Swap rows in A
            A[k], A[max_row] = A[max_row], A[k]

            # Swap corresponding elements in b
            b[k], b[max_row] = b[max_row], b[k]
        print(f"After {k} swap")
        # Elimination
        for i in range(k + 1, n):
            factor = A[i][k] / A[k][k]
            # print("What is the factor: ",factor)

            for j in range(k + 1, n):
                A[i][j] = A[i][j] - factor * A[k][j]
            A[i][k] = 0
            b[i] = b[i] - factor * b[k]
            print(f"    -- Row operation in row: {i+1}")
            printMatrix(A)

    # Back Substitution
    x = [0] * n

    x[n - 1] = b[n - 1] / A[n - 1][n - 1]

    for k in range(n - 2, -1, -1):
        sum_ax = 0
        for j in range(k + 1, n):
            sum_ax += A[k][j] * x[j]
        x[k] = (b[k] - sum_ax) / A[k][k]
    return x
n = int(input("Enter the size of the system: "))
print("Enter the coefficient matrix A:")
A = []
for i in range(n):
    row = list(map(float, input().split()))
    A.append(row)
print("Enter the RHS vector b:")
b = list(map(float, input().split()))
solution = gauss_elimination(n, A, b)
```

```
print("\nSolution Vector:")
for i in range(n):
    print(f"x{i+1} = {solution[i]:.2f}")
```

OUTPUT

Enter the size of the system: 3
Enter the coefficient matrix A:
2 1 -1
-3 -1 2
-2 1 2
Enter the RHS vector b:
8 -11 -3

3.JACOBI

```
def gauss_jacobi(A,b,x_old,TOL,KMAX):
    n=len(A)
    for i in range(n):
        print(f"Old x[{i}] = {x_old[i]}")
    TAG = True
    for k in range(1,KMAX+1):
        if TAG==False:
            break
        x_new = [0]*n
        for i in range(n):
            summ=0
            for j in range(n):
                if j!=i:
                    summ+=A[i][j]*x_old[j]
            x_new[i]=(b[i]-summ)/A[i][i]
            print(f"x_new[{i}] = {x_new[i]}")
        error = max(abs(x_new[i]-x_old[i]) for i in range(n))
        print(f"Iteration {k}: {x_new}")
        print(f"Error: {error}")
        if error<=TOL:
            TAG=False
        else:
            for i in range(n):
                x_old[i]=x_new[i]
```

```
    return x_new
n = int(input("Enter no. of equ: "))
A=[]
print("Enter the coeff. matrix: ")
for i in range(n):
    row = list(map(float,input(f"Row {i+1}: ").split()))
    A.append(row)
b = list(map(float,input("Enter RHS value: ").split()))
x0=list(map(float,input("Enter inital guess values: ").split()))
TOL=float(input("Enter tolerance value: "))
KMAX=int(input("Etner maximum no. of iteration: "))
ans = gauss_jacobi(A,b,x0,TOL,KMAX)
print("\nFinal Answer: ")
for i in range(n):
    print(f"x{i+1} = {ans[i]}")
```

OUTPUT

Enter no. of equ: 3
Enter the coeff. matrix:
Row 1: 10 1 1
Row 2: 2 10 1
Row 3: 2 2 10
Enter RHS value: 12 13 14
Enter inital guess values: 0 0 0
Enter tolerance value: 0.001
Etner maximum no. of iteration: 20

4.SEIDAL

```
def gauss_seidal(A,b,x_old,TOL,KMAX):
    n=len(b)
    for k in range(1,KMAX+1):
        x_new = x_old.copy()
        for i in range(n):
            summ=0
            for j in range(i):
                summ+=A[i][j]*x_new[j]
            for j in range(i+1,n):
                summ+=A[i][j]* x_old[j]
```

```
            x_new[i]=(b[i]-summ)/A[i][i]
        error = max(abs(x_new[i]-x_old[i]) for i in range(n))
        print(f"Iteration {k}: {x_new}")
        print(f"Error: {error}")
        if error<=TOL:
            break
        x_old=x_new.copy()
    return x_new
n=int(input("Enter no.of equ: "))
A=[]
print("Enter coeff. matrix: ")
for i in range(n):
    row = list(map(float,input(f"Row {i+1}: ").split()))
    A.append(row)
b = list(map(float,input("Enter RHS value: ").split()))
x0=list(map(float,input("Enter inital guess values: ").split()))
TOL=float(input("Enter tolerance value: "))
KMAX=int(input("Enter maximum no. of iteration: "))
ans = gauss_seidal(A,b,x0,TOL,KMAX)
print("\nFinal Answer: ")
for i in range(n):
    print(f"x{i+1} = {ans[i]}")
```

OUTPUT

Enter no.of equ: 3
Enter coeff. matrix:
Row 1: 10 1 1
Row 2: 2 10 1
Row 3: 2 2 10
Enter RHS value: 12 13 14
Enter inital guess values: 0 0 0
Enter tolerance value: 0.001
Enter maximum no. of iteration: 20

5.POWER METHOD

```
import math
MAX = 10
def main():
    n = int(input("Enter order of matrix: "))
    print("Enter matrix:")
```

```
A = []
for i in range(n):
    row = list(map(float, input().split()))
    A.append(row)
print("Enter initial vector:")
V = list(map(float, input().split()))
tol = float(input("Enter tolerance: "))
kmax = int(input("Enter maximum iterations: "))
lambda_val = 0.0
tag = True
W = [0.0] * n
for k in range(1, kmax + 1):
    if not tag:
        break
    for i in range(n):
        W[i] = 0.0
        for j in range(n):
            W[i] = W[i] + A[i][j] * V[j]
    c1 = W[0]
    for i in range(n):
        V[i] = W[i] / c1
    lambda_error = lambda_val - c1
    if lambda_error < 0:
        lambda_error = -lambda_error
    total_sum = 0.0
    print(f"\nIteration {k}")
    print(f"Eigen Value = {lambda_val}")
    print("Eigen Vector: ", end=" ")
    for i in range(n):
        print(f"{V[i]}", end=" ")
    print()
    if lambda_error <= tol:
        tag = False
    lambda_val = c1
print(f"\nDominant Eigen Value = {lambda_val}")
print("Dominant Eigen Vector = ", end=" ")
for i in range(n):
    print(f"{V[i]}", end=" ")
print()
```

```
if __name__ == "__main__":
    main()
```

OUTPUT

Enter order of matrix: 2

Enter matrix:

4 1

2 3

Enter initial vector:

1 1

Enter tolerance: 0.001

Enter maximum iterations: 20

6.JORDAN INVERSE

```
n = int(input("Enter the size of matrix: "))
A = []
for i in range(n):
    row = list(map(float, input().split()))
    A.append(row)
aug = []
for i in range(n):
    identity = [1 if i == j else 0 for j in range(n)]
    aug.append(A[i] + identity)
print("\nInitial Augmented Matrix:")
for row in aug:
    print(" ".join(f"{x:.3f}" for x in row))
for i in range(n):
    if aug[i][i] == 0:
        for k in range(i + 1, n):
            if aug[k][i] != 0:
                aug[i], aug[k] = aug[k], aug[i]
                break
        else:
            print("Inverse does not exist")
            exit()
    pivot = aug[i][i]
```

```
    for j in range(2 * n):
        aug[i][j] /= pivot
    for k in range(n):
        if k != i:
            factor = aug[k][i]
            for j in range(2 * n):
                aug[k][j] -= factor * aug[i][j]
    print(f"\nIteration {i+1}:")
    for row in aug:
        print(" ".join(f"{x:.3f}" for x in row))
print("\nInverse matrix:")
for i in range(n):
    print(" ".join(f"{aug[i][j]:.3f}" for j in range(n, 2 * n)))
print("\nA * A-inverse:")
result = []
for i in range(n):
    row = []
    for j in range(n):
        total = 0
        for k in range(n):
            total += A[i][k] * aug[k][n+j]
        row.append(total)
    result.append(row)
for row in result:
    print(" ".join(f"{x:.3f}" for x in row))
```

OUTPUT

Enter the size of matrix: 3

1 2 3

0 1 4

5 6 0

7.LAGRANGE

```
import sympy as sp
n = int(input("Enter the number of points: "))
l = []
for i in range(n):
    row = list(map(float, input().split()))
    l.append(row)
```

```
x_value = float(input("Enter x value: "))
x = sp.Symbol('x')
value_count=0
sum = 0
term2 = 0
for i in range(n):
    p = 1
    term1 = l[i][1]
    for j in range(n):
        if j!=i:
            term1*=(x-l[j][0])/(l[i][0]-l[j][0])
            p*=(x_value-l[j][0])/(l[i][0]-l[j][0])
            value_count+=4

    sum+=p*l[i][1]
    value_count+=2
    term2+=term1*l[i][1]
poly_form = sp.expand(term2)
degree = sp.degree(poly_form, x)
print(x, x_value)
print("\nLagrange form : ",term2)
print("\nPolynomial form: ",poly_form)
print(f"\nInterpolation of {x_value} :",sum)
print("\nNo of operations to get the value: ",value_count)
print("\nNo of operations to get the polynomial eqn: ",sp.count_ops(poly_form))
print("\nNo of operations to interpolate at x: ",2*degree)
```

OUTPUT

Enter the number of points: 3

1 1

2 4

3 9

Enter x value: 2.5

8.NEWTON DIVIDED DIFF

```
n = int(input("Enter number of data points: "))
x = []
```

```
y = []
print("Enter x and y values:")
for i in range(n):
    xi = float(input(f"x[{i}] = "))
    yi = float(input(f"y[{i}] = "))
    x.append(xi)
    y.append(yi)
d = [[0.0 for j in range(n)] for i in range(n)]
for j in range(n):
    d[0][j] = y[j]
for i in range(1, n):
    for j in range(n - i):
        d[i][j] = (d[i-1][j+1] - d[i-1][j]) / (x[j+i] - x[j])
print("\nDivided Difference Table:")
header = f"{'x':<10}{'y':<10}"
for i in range(1, n):
    header += f"{f'dd{i}':<10}"
print(header)
print("-" * (10 * (n + 1)))
for j in range(n):
    row_str = f"{x[j]:<10.4f}{d[0][j]:<10.4f}"
    for i in range(1, n - j):
        row_str += f"{d[i][j]:<10.4f}"
    print(row_str)
equation = f"{d[0][0]}"
for i in range(1, n):
    equation += f" + ({d[i][0]})"
    for j in range(i):
        equation += f"(x - {x[j]})"
print("\nNewton Interpolation Equation:")
print("P(x) =", equation)
x_value = float(input("\nEnter x value: "))
y_interp = d[0][0]
p = 1
for i in range(n - 1):
    p = p * (x_value - x[i])
    y_interp += p * d[i+1][0]
print(f"\nValue at x = {x_value}: {y_interp}")
```

OUTPUT

Enter number of data points: 3

Enter x and y values:

x[0] = 1

y[0] = 1

x[1] = 2

y[1] = 4

x[2] = 3

y[2] = 9

Enter x value: 2.5

9.FORWARD DIFF

```
n = int(input("Enter number of data points: "))
x = []
y = []
print("Enter x and y values:")
for i in range(n):
    xi = float(input(f"x[{i}] = "))
    yi = float(input(f"y[{i}] = "))
    x.append(xi)
    y.append(yi)
h = x[1] - x[0]
for i in range(2, n):
    if abs((x[i] - x[i-1]) - h) > 1e-10:
        print("Error: x values must be equally spaced.")
        exit()
d = [[0.0 for j in range(n)] for i in range(n)]
for j in range(n):
    d[0][j] = y[j]
for i in range(1, n):
    for j in range(n - i):
        d[i][j] = d[i-1][j+1] - d[i-1][j]
print("\nForward Difference Table:")
header = f"{'x':<10}{'y':<10}"
for i in range(1, n):
    header += f"{f'dd{i}':<10}"
print(header)
```

```
print("-" * (10 * (n + 1)))
for j in range(n):
    row_str = f"{x[j]:<10.4f}{y[j]:<10.4f}"
    for i in range(1, n):
        if j >= i:
            row_str += f"{d[i][j-i]:<10.4f}"
        else:
            row_str += f"{'':<10}"
    print(row_str)
equation = f"{d[0][0]}"
for i in range(1, n):
    equation += f" + ({d[i][0]})"
    for j in range(i):
        equation += f"(u - {j})"
    equation += f"/{i}!"
print("\nNewton Forward Interpolation Equation:")
print("P(x) =", equation)
l = int(input("\nHow many x values do you want to interpolate? "))
for k in range(l):
    x_value = float(input(f"\nEnter x value {k+1}: "))
    u = (x_value - x[0]) / h
    y_interp = d[0][0]
    u_product = 1
    factorial = 1
    for i in range(1, n):
        u_product = u_product * (u - (i - 1))
        factorial = factorial * i
        y_interp += (u_product / factorial) * d[i][0]
    print(f"u = (x - x0) / h = ({x_value} - {x[0]}) / {h} = {u}")
    print(f"Value at x = {x_value}: {y_interp}")
```

OUTPUT

Enter number of data points: 3

Enter x and y values:

x[0] = 1

y[0] = 1

x[1] = 2

y[1] = 4

x[2] = 3

y[2] = 9

Enter x value: 2.5

10.BACKWARD DIFF

```
import matplotlib.pyplot as plt
n = int(input("Enter number of data points: "))
x = []
y = []
print("Enter x and y values:")
for i in range(n):
    xi = float(input(f"x[{i}] = "))
    yi = float(input(f"y[{i}] = "))
    x.append(xi)
    y.append(yi)
h = x[1] - x[0]
for i in range(2, n):
    if abs((x[i] - x[i-1]) - h) > 1e-10:
        print("Error: x values must be equally spaced.")
        exit()
d = [[0.0 for j in range(n)] for i in range(n)]
for j in range(n):
    d[0][j] = y[j]
for i in range(1, n):
    for j in range(n - i):
        d[i][j] = d[i-1][j+1] - d[i-1][j]
print("\nBackward Difference Table:")
header = f"{'x':<10}{'y':<10}"
for i in range(1, n):
    header += f"{f'dd{i}':<10}"
print(header)
print("-" * (10 * (n + 1)))
for j in range(n):
    row_str = f"{x[j]:<10.4f}{y[j]:<10.4f}"
    for i in range(1, n):
        if j >= i:
            row_str += f"{d[i][j-i]:<10.4f}"
        else:
            row_str += f"{'':<10}"
    print(row_str)
```

```
equation = f"{d[0][n-1]}"
for i in range(1, n):
    equation += f" + ({d[i][n-i-1]})"
    for j in range(i):
        equation += f"(u + {j})"
    equation += f"/{i}!"
print("\nNewton Backward Interpolation Equation:")
print("P(x) =", equation)
l = int(input("\nHow many x values do you want to interpolate? "))
x_inter = []
y_inter = []
for k in range(l):
    x_value = float(input(f"\nEnter x value {k+1}: "))
    u = (x_value - x[n-1]) / h
    y_interp = d[0][n-1]
    u_product = 1
    factorial = 1
    for i in range(1, n):
        u_product = u_product * (u + (i - 1))
        factorial = factorial * i
        y_interp += (u_product / factorial) * d[i][n-i-1]
    x_inter.append(x_value)
    y_inter.append(y_interp)
    print(f"u = (x - xn) / h = ({x_value} - {x[n-1]}) / {h} = {u}")
    print(f"Value at x = {x_value}: {y_interp}")
plt.plot(x, y, 'bo-', label='Given Data')
plt.plot(x_inter, y_inter, 'go', label='Interpolated Points')
plt.xlabel("x")
plt.ylabel("y")
plt.title("Newton Backward Interpolation")
plt.legend()
plt.grid(True)
plt.show()B
```

OUTPUT

Enter number of data points: 3

Enter x and y values:

x[0] = 1

y[0] = 1

x[1] = 2

y[1] = 4
x[2] = 3
y[2] = 9
How many x values do you want to interpolate? 2
Enter x value 1: 2.5
u = (x - xn) / h = (2.5 - 3.0) / 1.0 = -0.5
Value at x = 2.5: 6.25
Enter x value 2: 2.8
u = (x - xn) / h = (2.8 - 3.0) / 1.0 = -0.20000000000000018
Value at x = 2.8: 7.84

11.CUBIC SPLINE

```
import sympy as sp
n = int(input("Enter number of data points: "))
x_data = []
y_data = []
for i in range(n):
    x_val, y_val = map(float, input(f"Enter x[{i}] and y[{i}]: ").split())
    x_data.append(x_val)
    y_data.append(y_val)
l = int(input("Enter number of interpolated values: "))
x_values = []
for k in range(l):
    while True:
        x_val = float(input(f"Enter interpolation x[{k}]: "))
        if x_data[0] <= x_val <= x_data[-1]:
            x_values.append(x_val)
            break
        else:
            print("Incorrect interpolated value. Enter again.")
x = sp.Symbol('x')
h=[0]
for i in range (1,n):
    h.append(x_data[i]-x_data[i-1])
m = [0] * n
for i in range(1, n - 1):
    m[i] = sp.Symbol(f'm{i}')
equation = []
```

```
for i in range(1, n - 1):
    eq = (
        m[i - 1]
        + 4 * m[i]
        + m[i + 1]
        - (6 / h[i]**2) *
        (y_data[i + 1] - 2 * y_data[i] + y_data[i - 1])
    )
    equation.append(eq)
unknowns = m[1:n-1]
values = sp.solve(equation, unknowns)
for i in range(1, n - 1):
    m[i] = values[m[i]]
fx = []
for i in range(1, n):
    eq = (
        ((x_data[i] - x)**3 * m[i - 1]) / (6 * h[i])
        + ((x - x_data[i - 1])**3 * m[i]) / (6 * h[i])
        + ((x_data[i] - x) / h[i]) *
          (y_data[i - 1] - (h[i]**2 * m[i - 1]) / 6)
        + ((x - x_data[i - 1]) / h[i]) *
          (y_data[i] - (h[i]**2 * m[i]) / 6)
    )
    fx.append(sp.expand(eq))
print("\nCubic Spline Functions:")
for i in range(n - 1):
    print(f"F{i + 1}(x) =", fx[i])
print("\nInterpolated Values:")
for x_val in x_values:
    interval = None
    for i in range(n - 1):
        if x_data[i] <= x_val <= x_data[i + 1]:
            interval = i
            break
    if interval is not None:
        result = fx[interval].subs(x, x_val)
        result = float(result)
        print(f"S({x_val}) = {result}")
plot=[]
```

```
for i in range (n-1):
    p=sp.plot(fx[i],(x,x_data[i],x_data[i+1]),show=False,label=f'F{i+1}X')
    plot.append(p)
final_plot=plot[0]
for p in plot[1:]:
    final_plot.extend(p)
final_plot.show()
```

OUTPUT

Enter number of data points: 3
Enter x[0] and y[0]: 1 1
Enter x[1] and y[1]: 2 4
Enter x[2] and y[2]: 3 9
Enter number of interpolated values: 2
Enter interpolation x[0]: 1.5
Enter interpolation x[1]: 2.5

12.TRAPEZOIDAL

```
import sympy as sp
n = int(input("Enter number of subintervals: "))
a = float(input("Enter lower limit a: "))
b = float(input("Enter upper limit b: "))
function_input = input("Enter the function f(x): ")
x = sp.Symbol('x')
f_expr = sp.sympify(function_input)
f = sp.lambdify(x, f_expr, 'math')
h = (b - a) / n
sum_value = f(a) + f(b)
for i in range(1, n):
    x_value = a + i * h
    sum_value += 2 * f(x_value)
integral = (h / 2) * sum_value
exact_integral = sp.integrate(f_expr, (x, a, b))
exact_value = float(exact_integral)
error = abs(exact_value - integral)
print("\nFunction f(x) =", f_expr)
print("Step size h =", h)
print("Approximate integral =", integral)
```

```
print("Exact integral =", exact_value)
print("Error =", error)
```

OUTPUT

Enter number of subintervals: 4

Enter lower limit a: 0

Enter upper limit b: 2

Enter the function f(x): x2**

13.SIMPSON

```
import sympy as sp
n=int(input("Number of Sub-Intervals:"))
a=int(input("Start Interval: "))
b=int(input("End Interval: "))
h= (b-a)/n
x = sp.Symbol('x')
f = sp.sympify(input("Enter f(x): "))
area = f.subs(x,a) + f.subs(x,b)
for i in range (1,n):
    m= a +(i*h)
    if i%2==1:
        area += 4*(f.subs(x,m))
    else:
        area += 2*(f.subs(x,m))
area = area * (h/3)
print("The Area under the Curve (Using Composite Simson's 1/3 Rule): ",area)
integral_area=sp.integrate(f,(x,a,b))
print("The Area under the Curve (Using Integration): ",float(integral_area))
print("Error (Difference between Exact Integral Value and Using Composite Simson's 1/3 Rule)
: ",(float(integral_area)-area))
```

OUTPUT

Number of Sub-Intervals: 4

Start Interval: 0

End Interval: 2

Enter f(x): x2**

14.QUADRATURE

```
import sympy as sp
n=int(input("Number of Sub-Intervals: "))
a=int(input("Start Interval: "))
b=int(input("End Interval: "))
h= (b-a)/n
x = sp.Symbol('x')
f = sp.sympify(input("Enter f(x): "))
area = 0
d = h/(2*(3**0.5))
for i in range(n):
    xi = a + (i*h)
    xj = xi + h
    midpoint = (xi+xj)/2
    x1 = midpoint - d
    x2 = midpoint + d
    area += f.subs(x,x1) + f.subs(x,x2)
area = area * (h/2)
print("The Area under the Curve (Using Composite Two Point Gauss Quadrature Rule): ",area)
integral_area=sp.integrate(f,(x,a,b))
print("The Area under the Curve (Using Integration): ",float(integral_area))
print("Error (Difference between Exact Integral Value and Using Composite Two Point Gauss
Quadrature Rule) : ",(float(integral_area)-area))
```

OUTPUT

Number of Sub-Intervals: 4

Start Interval: 0

End Interval: 2

Enter f(x): x2**

15.ADAM BASHFORTH

```
f = input("Enter f(x, y) (e.g. x + y): ")
x = []
y = []
print("Enter the 4 starting values (x y), one pair per line:")
for _ in range(4):
    a, b = map(float, input().split())
    x.append(a)
```

```
    y.append(b)
X_req = float(input("Enter the value of x at which y is required: "))
h = x[1] - x[0]
n = round((X_req - x[3]) / h)
print("\nAdams-Bashforth-Moulton Predictor-Corrector Results")
for i in range(3, 3 + n):
    F = [eval(f, {"x": x[j], "y": y[j]}) for j in range(i - 3, i + 1)]
    X_k = x[i] + h
    Y_p = y[i] + (h / 24) * (55 * F[3]- 59 * F[2]+ 37 * F[1]- 9 * F[0])
    print(f"\nFor X = {X_k:.6f}")
    print(f"Predicted Y = {Y_p:.6f}")
    iteration = 0
    while True:
        iteration += 1
        Y_c = y[i] + (h / 24) * (9 * eval(f, {"x": X_k, "y": Y_p})+ 19 * F[3]- 5 * F[2]+ F[1])
        print(f"Correction {iteration}: Y = {Y_c:.6f}")
        if abs(Y_c - Y_p) < 1e-6:
            break
        Y_p = Y_c
    x.append(X_k)
    y.append(Y_c)
    print(f"Converged Y = {Y_c:.6f}")
    print(f"Number of iterations = {iteration}")
import matplotlib.pyplot as plt
plt.figure(figsize=(10,5))
plt.plot(x,y)
plt.show()
```

OUTPUT

Enter f(x, y) (e.g. x + y): x2 + 2*y**

Enter the 4 starting values (x y), one pair per line:

0 1

0.1 1.2214

0.2 1.4932

0.3 1.8279

Enter the value of x at which y is required: 0.5

16.MILNE

```
f = input("Enter f(x, y) (e.g. x + y): ")
x = []
y = []
print("Enter the 4 starting values (x y), one pair per line:")
for _ in range(4):
    a, b = map(float, input().split())
    x.append(a)
    y.append(b)
X_req = float(input("Enter the value of x at which y is required: "))
h = x[1] - x[0]
n = round((X_req - x[3]) / h)
print("\n--- Milne's Predictor-Corrector Results ---")
for i in range(3, 3 + n):
    F = [
        eval(f, {"x": x[j], "y": y[j]})
        for j in range(i - 3, i + 1)
    ]
    X_k = x[i] + h
    # Predictor
    Y_p = y[i - 3] + (4 * h / 3) * (
        2 * F[1] - F[2] + 2 * F[3]
    )
    print(f"\nFor X = {X_k:.6f}")
    print(f"Predicted Y = {Y_p:.6f}")
    iteration = 0
    # Repeated correction
    while True:
        iteration += 1
        Y_c = y[i - 1] + (h / 3) * (
            F[2]
            + 4 * F[3]
            + eval(f, {"x": X_k, "y": Y_p})
        )
        print(f"Correction {iteration}: Y = {Y_c:.6f}")
        if abs(Y_c - Y_p) < 1e-6:
            break
        Y_p = Y_c
    x.append(X_k)
    y.append(Y_c)
```

```
    print(f"Converged Y = {Y_c:.6f}")
    print(f"Number of iterations = {iteration}")
import matplotlib.pyplot as plt
plt.figure(figsize=(10,5))
plt.plot(x,y)
plt.show()
```

OUTPUT

Enter f(x, y) (e.g. x + y): x2 + 2*y**
Enter the 4 starting values (x y), one pair per line:
0 1
0.1 1.2214
0.2 1.4932
0.3 1.8279
Enter the value of x at which y is required: 0.5

17.RUNGE KUTTA

```
f = input("Enter f(x, y) (e.g. x + y): ")
x = float(input("Enter the initial value of x (x0): "))
y = float(input("Enter the initial value of y (y0): "))
h = float(input("Enter the step size (h): "))
X_req = float(input("Enter the value of x at which y is required: "))
x_val=[]
y_val=[]
n = round((X_req - x) / h)
print("\n--- Runge-Kutta 4th Order Results ---")
for _ in range(n):
    k1 = h * eval(f, {"x": x, "y": y})
    k2 = h * eval(f, {"x": x + h / 2, "y": y + k1 / 2})
    k3 = h * eval(f, {"x": x + h / 2, "y": y + k2 / 2})
    k4 = h * eval(f, {"x": x + h, "y": y + k3})
    y = y + (k1 + 2 * k2 + 2 * k3 + k4) / 6
    x = x + h
    x_val.append(x)
    y_val.append(y)
    print(f"X = {x:.6f}, Y = {y:.6f}")
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(10,5))
plt.plot(x_val,y_val)
plt.show()
```

OUTPUT

Enter f(x, y) (e.g. x + y): x+y
Enter the initial value of x (x0): 0
Enter the initial value of y (y0): 1
Enter the step size (h): 0.1
Enter the value of x at which y is required: 0.5

18.FINITE DIFF FOR ELIPTICAL-method(1)

```
# 20. Finite Difference Method for Elliptic PDE (Laplace & Poisson Equations)
# Solves ∇²u = 0 (or ∇²u = f(x, y)) using 5-point stencil & Liebmann's method
print("=== Elliptic PDE Solver: Laplace / Poisson Equation ===")
print("Standard 5-point formula: u(i, j) = 0.25 * [u(i+1, j) + u(i-1, j) + u(i, j+1) + u(i, j-1)]")
nx = int(input("\nEnter number of intervals in x-direction (nx): "))
ny = int(input("Enter number of intervals in y-direction (ny): "))
print("\nEnter Dirichlet Boundary Conditions:")
top = float(input("Top boundary value: "))
bottom = float(input("Bottom boundary value: "))
left = float(input("Left boundary value: "))
right = float(input("Right boundary value: "))
tol = float(input("Enter convergence tolerance (e.g. 1e-4): "))
max_iter = int(input("Enter maximum iterations (e.g. 200): "))
# Initialize grid (ny + 1 rows, nx + 1 columns)
# grid[j][i]: j is y index (0 to ny), i is x index (0 to nx)
grid = [[0.0 for _ in range(nx + 1)] for _ in range(ny + 1)]
# Apply boundary conditions
for i in range(nx + 1):
    grid[0][i] = bottom
    grid[ny][i] = top
for j in range(ny + 1):
    grid[j][0] = left
    grid[j][nx] = right
# Corner averages
grid[0][0] = (bottom + left) / 2.0
grid[0][nx] = (bottom + right) / 2.0
```

```
grid[ny][0] = (top + left) / 2.0
grid[ny][nx] = (top + right) / 2.0
# Initialize interior nodes with average of boundaries
avg_val = (top + bottom + left + right) / 4.0
for j in range(1, ny):
    for i in range(1, nx):
        grid[j][i] = avg_val
print("\nStarting Liebmann's (Gauss-Seidel) iterations...")
converged = False
for it in range(1, max_iter + 1):
    max_error = 0.0
    for j in range(ny - 1, 0, -1):  # top-down row order
        for i in range(1, nx):
            old = grid[j][i]
            # Standard 5-point stencil
            grid[j][i] = 0.25 * (grid[j + 1][i] + grid[j - 1][i] + grid[j][i + 1] + grid[j][i - 1])
            err = abs(grid[j][i] - old)
            if err > max_error:
                max_error = err
    if it % 10 == 0 or it == 1:
        print(f"Iteration {it:3d}: Max Error = {max_error:.6f}")
    if max_error < tol:
        print(f"Converged successfully in {it} iterations! (Max Error = {max_error:.6e})")
        converged = True
        break
if not converged:
    print(f"Completed {max_iter} iterations.")
print("\n--- Final Grid Values u[y][x] ---")
print("    " + "".join(f"  x={i:<5d}" for i in range(nx + 1)))
print("    " + "-" * ((nx + 1) * 8))
for j in range(ny, -1, -1):  # print from top (ny) down to bottom (0)
    row_str = "".join(f"{grid[j][i]:8.3f}" for i in range(nx + 1))
    print(f"y={j:<2d}|{row_str}")
```

OUTPUT

Enter number of intervals in x-direction (nx): 2

Enter number of intervals in y-direction (ny): 2

Enter Dirichlet Boundary Conditions:

Top boundary value: 100

Bottom boundary value: 0
Left boundary value: 0
Right boundary value: 0
Enter convergence tolerance (e.g. 1e-4): 0.0001
Enter maximum iterations (e.g. 200): 100

18.FINITE DIFF FOR ELIPTICAL-method(2)

```
import numpy as np
N = int(input("Enter number of intervals in x-direction (N): "))
M = int(input("Enter number of intervals in y-direction (M): "))
tolerance = float(input("Enter tolerance: "))
max_iterations = int(input("Enter maximum iterations: "))
a = float(input("Enter starting value of x (a): "))
b = float(input("Enter ending value of x (b): "))
c = float(input("Enter starting value of y (c): "))
d = float(input("Enter ending value of y (d): "))
h = (b - a) / N
k = (d - c) / M
print("\nStep 1: Mesh")
print("h =", h)
print("k =", k)
U = np.zeros((M + 1, N + 1))
# Boundary conditions
bottom = float(input("Enter bottom boundary value: "))
top = float(input("Enter top boundary value: "))
left = float(input("Enter left boundary value: "))
right = float(input("Enter right boundary value: "))
U[0, :] = bottom
U[:, 0] = left
U[:, N] = right
U[M, 1:N] = top
# Corner values
U[0, 0] = (bottom + left) / 2
U[0, N] = (bottom + right) / 2
U[M, 0] = (top + left) / 2
U[M, N] = (top + right) / 2
print("\nInitial Grid:")
```

```
for row in U[::-1]:
    print(" ".join(f"{x:10.4f}" for x in row))
print("\nGAUSS-SEIDEL ITERATIONS")
for iteration in range(1, max_iterations + 1):
    old_U = U.copy()
    print(f"\nIteration {iteration}")
    for j in range(1, M):
        for i in range(1, N):
            U[j, i] = (
                U[j, i + 1]
                + U[j, i - 1]
                + U[j + 1, i]
                + U[j - 1, i]
            ) / 4.0
            print(
                f"u({i},{j}) = "
                f"1/4 ["
                f"{U[j, i + 1]:.6f} + "
                f"{U[j, i - 1]:.6f} + "
                f"{U[j + 1, i]:.6f} + "
                f"{U[j - 1, i]:.6f}"
                f"] = {U[j, i]:.6f}"
            )
    error = np.max(np.abs(U - old_U))
    print(f"\nMaximum change E = {error:.6f}")
    if error < tolerance:
        print("\nConvergence achieved!")
        print(f"Stopping at iteration {iteration}")
        break
print("\nFINAL APPROXIMATE SOLUTION")
for row in U[::-1]:
    print(" ".join(f"{x:10.6f}" for x in row))
print("\nFinal maximum error =", f"{error:.6f}")
print("Number of iterations =", iteration)
```

OUTPUT

```
Enter number of intervals in x-direction (N): 4
Enter number of intervals in y-direction (M): 4
Enter tolerance: 0.01
Enter maximum iterations: 100
```

Enter starting value of x (a): 0
Enter ending value of x (b): 4
Enter starting value of y (c): 0
Enter ending value of y (d): 4
Step 1: Mesh
h = 1.0
k = 1.0
Enter bottom boundary value: 0
Enter top boundary value: 100
Enter left boundary value: 0
Enter right boundary value: 0

19.FINITE DIFFERENCE PARABOLIC(METHOD-1)

```
# 21. Finite Difference Method for Parabolic PDE (1D Heat Equation)
# Solves ∂u/∂t = c² ∂²u/∂x² using Bender-Schmidt Explicit Scheme
import math
print("=== Parabolic PDE Solver: 1D Heat Equation (Bender-Schmidt) ===")
print("Formula: u(i, j+1) = λ * u(i-1, j) + (1 - 2λ) * u(i, j) + λ * u(i+1, j)")
print("Stability condition: λ = (c² * k) / h² <= 0.5\n")
length = float(input("Enter length of rod (L): "))
c2 = float(input("Enter thermal diffusivity c² (e.g. 1.0): "))
nx = int(input("Enter number of spatial divisions (nx): "))
nt = int(input("Enter number of time steps (nt): "))
total_time = float(input("Enter total time duration (T): "))
h = length / nx
k = total_time / nt
lam = (c2 * k) / (h ** 2)
print(f"\nGrid spacing: h = {h:.4f}, time step: k = {k:.4f}")
print(f"Mesh ratio λ = {lam:.4f}")
if lam > 0.5 + 1e-9:
    print(f"WARNING: λ = {lam:.4f} > 0.5. The explicit scheme is unstable! Consider increasing
nx or nt.")
f_str = input("\nEnter initial condition f(x) for u(x, 0) (press Enter for sin(pi*x/L)): ").strip()
if not f_str:
    f_init = lambda x: math.sin(math.pi * x / length)
else:
    f_init = lambda x: eval(f_str, {"x": x, "math": math, "sin": math.sin, "cos": math.cos, "pi":
math.pi, "exp": math.exp})
```

```
left_bc = float(input("Enter boundary condition at x = 0 (left): "))
right_bc = float(input("Enter boundary condition at x = L (right): "))
# u[j][i]: row j is time level (0 to nt), column i is spatial node (0 to nx)
u = [[0.0 for _ in range(nx + 1)] for _ in range(nt + 1)]
# Initial condition at t = 0
for i in range(nx + 1):
    u[0][i] = f_init(i * h)
# Boundary conditions for all time steps
for j in range(nt + 1):
    u[j][0] = left_bc
    u[j][nx] = right_bc
# Time marching
for j in range(nt):
    for i in range(1, nx):
        u[j + 1][i] = lam * u[j][i - 1] + (1.0 - 2.0 * lam) * u[j][i] + lam * u[j][i + 1]
print("\n--- Temperature Distribution Table u(x, t) ---")
header = f"{'Time':<10}" + "".join(f"{f'x={i*h:.2f}':<10}" for i in range(nx + 1))
print(header)
print("-" * len(header))
for j in range(nt + 1):
    row_str = f"{j * k:<10.4f}" + "".join(f"{u[j][i]:<10.4f}" for i in range(nx + 1))
    print(row_str)
print("-" * len(header))
print(f"Final temperature distribution at t = {total_time:.4f} reached.")
```

OUTPUT

Enter length of rod (L): 2
Enter thermal diffusivity c² (e.g. 1.0): 1
Enter number of spatial divisions (nx): 2
Enter number of time steps (nt): 4
Enter total time duration (T): 1
Enter initial condition f(x) for u(x, 0) (press Enter for sin(pi*x/L)): x*(2-x)
Enter boundary condition at x = 0 (left): 0
Enter boundary condition at x = L (right): 0

19.FINITE DIFFERENCE PARABOLIC(METHOD-2)-FTCS METHOD

```
import math
L = float(input("Enter length L: "))
T = float(input("Enter total time T: "))
```

```
alpha = float(input("Enter thermal diffusivity alpha: "))
M = int(input("Enter spatial intervals M: "))
N = int(input("Enter time steps N: "))
if L <= 0 or T <= 0 or alpha <= 0 or M < 2 or N < 1:
    print("Invalid input.")
else:
    h = L / M
    k = T / N
    r = alpha * k / (h ** 2)
    print("\nSpatial step h =", h)
    print("Time step k =", k)
    print("Parameter r =", r)
    if r > 0.5:
        print("Stability condition violated.")
    else:
        print("Stability condition satisfied.")
        x = [i * h for i in range(M + 1)]
        U = [
            100 * math.sin(math.pi * x[i] / L)
            for i in range(M + 1)
        ]
        U[0] = 0.0
        U[M] = 0.0
        print("\nInitial temperature distribution:")
        print("Position\tTemperature")
        for i in range(M + 1):
            print(f"{x[i]:.4f}\t\t{U[i]:.4f}")
        for n in range(N):
            U_new = U.copy()
            for i in range(1, M):
                U_new[i] = (
                    r * U[i - 1]
                    + (1 - 2 * r) * U[i]
                    + r * U[i + 1]
                )
            U_new[0] = 0.0
            U_new[M] = 0.0
            U = U_new
            print(f"\nTime step {n + 1}, t = {(n + 1) * k:.4f}")
```

```
            for i in range(M + 1):
                print(f"U({x[i]:.2f}) = {U[i]:.4f}")
        print("\nFinal solution:")
        for i in range(M + 1):
            print(f"U({x[i]:.2f}) = {U[i]:.4f}")
```

OUTPUT

Enter length L: 2

Enter total time T: 1

Enter thermal diffusivity alpha: 1

Enter spatial intervals M: 2

Enter time steps N: 4

19.FINITE DIFFERENCE PARABOLIC(METHOD-3)-CRANK NICOLSON METHOD

```
import math
def thomas_algorithm(a, b, c, d):
    n = len(d)
    cp = [0.0] * n
    dp = [0.0] * n
    solution = [0.0] * n
    if abs(b[0]) < 1e-14:
        raise ValueError("Zero pivot encountered.")
    cp[0] = c[0] / b[0] if n > 1 else 0.0
    dp[0] = d[0] / b[0]
    for i in range(1, n):
        denominator = b[i] - a[i] * cp[i - 1]
        if abs(denominator) < 1e-14:
            raise ValueError("Zero pivot encountered.")
        if i < n - 1:
            cp[i] = c[i] / denominator
        dp[i] = (d[i] - a[i] * dp[i - 1]) / denominator
    solution[n - 1] = dp[n - 1]
    for i in range(n - 2, -1, -1):
        solution[i] = dp[i] - cp[i] * solution[i + 1]
    return solution

print("CRANK-NICOLSON METHOD")
L = float(input("Enter length L: "))
T = float(input("Enter total time T: "))
alpha = float(input("Enter thermal diffusivity alpha: "))
```

```
M = int(input("Enter spatial intervals M: "))
N = int(input("Enter time steps N: "))
if L <= 0 or T <= 0 or alpha <= 0 or M < 2 or N < 1:
    print("Invalid input.")
else:
    h = L / M
    k = T / N
    r = alpha * k / (h ** 2)
    print("\nSpatial step h =", h)
    print("Time step k =", k)
    print("Parameter r =", r)
    x = [i * h for i in range(M + 1)]
    U = [
        100 * math.sin(math.pi * x[i] / L)
        for i in range(M + 1)
    ]
    U[0] = 0.0
    U[M] = 0.0
    print("\nInitial temperature distribution:")
    for i in range(M + 1):
        print(f"U({x[i]:.2f}) = {U[i]:.4f}")
    p = M - 1
    a = [-r / 2] * p
    b = [1 + r] * p
    c = [-r / 2] * p
    print("\nTridiagonal matrix coefficients:")
    print("Lower diagonal =", -r / 2)
    print("Main diagonal  =", 1 + r)
    print("Upper diagonal =", -r / 2)
    for n in range(N):
        t_new = (n + 1) * k
        rhs = [0.0] * p
        for j in range(p):
            i = j + 1
            rhs[j] = (
                (r / 2) * U[i - 1]
                + (1 - r) * U[i]
                + (r / 2) * U[i + 1]
            )
```

```
        rhs[0] += (r / 2) * 0.0
        rhs[p - 1] += (r / 2) * 0.0
        interior = thomas_algorithm(a, b, c, rhs)
        U_new = [0.0] * (M + 1)
        for j in range(p):
            U_new[j + 1] = interior[j]
        U = U_new
        print(f"\nTime step {n + 1}, t = {t_new:.4f}")
        for i in range(M + 1):
            print(f"U({x[i]:.2f}) = {U[i]:.4f}")
    print("\nFinal temperature distribution:")
    for i in range(M + 1):
        print(f"U({x[i]:.2f}) = {U[i]:.4f}")
    print("\nCalculation completed successfully.")
```

OUTPUT

CRANK-NICOLSON METHOD

Enter length L: 2

Enter total time T: 1

Enter thermal diffusivity alpha: 1

Enter spatial intervals M: 2

Enter time steps N: 4

20.FINITE DIFF WITH HYPERBOLIC(METHOD1)

```
# 22. Finite Difference Method for Hyperbolic PDE (1D Wave Equation)
# Solves ∂²u/∂t² = c² ∂²u/∂x² using Explicit Finite Difference Scheme
import math
print("=== Hyperbolic PDE Solver: 1D Wave Equation (Vibrating String) ===")
print("Formula: u(i, j+1) = 2(1 - λ²) u(i, j) + λ² [u(i+1, j) + u(i-1, j)] - u(i, j-1)")
print("CFL stability condition: λ = (c * k) / h <= 1.0\n")
length = float(input("Enter length of string (L): "))
c = float(input("Enter wave speed (c): "))
nx = int(input("Enter number of spatial divisions (nx): "))
nt = int(input("Enter number of time steps (nt): "))
total_time = float(input("Enter total time duration (T): "))
h = length / nx
k = total_time / nt
```

```
lam = (c * k) / h
lam2 = lam ** 2
print(f"\nGrid spacing: h = {h:.4f}, time step: k = {k:.4f}")
print(f"Courant number λ = {lam:.4f}")
if lam > 1.0 + 1e-9:
    print(f"WARNING: λ = {lam:.4f} > 1.0. CFL stability condition violated! Numerical instability will occur.")
f_str = input("\nEnter initial displacement f(x) for u(x, 0) (press Enter for sin(pi*x/L)): ").strip()
if not f_str:
    f_disp = lambda x: math.sin(math.pi * x / length)
else:
    f_disp = lambda x: eval(f_str, {"x": x, "math": math, "sin": math.sin, "cos": math.cos, "pi": math.pi, "exp": math.exp})
g_str = input("Enter initial velocity g(x) = ∂u/∂t(x, 0) (press Enter for 0): ").strip()
if not g_str:
    g_vel = lambda x: 0.0
else:
    g_vel = lambda x: eval(g_str, {"x": x, "math": math, "sin": math.sin, "cos": math.cos, "pi": math.pi})
left_bc = float(input("Enter left boundary condition at x = 0: "))
right_bc = float(input("Enter right boundary condition at x = L: "))
# u[j][i]: row j is time level (0 to nt), column i is spatial node (0 to nx)
u = [[0.0 for _ in range(nx + 1)] for _ in range(nt + 1)]
# Initial displacement at t = 0
for i in range(nx + 1):
    u[0][i] = f_disp(i * h)
# Boundary conditions for all time levels
for j in range(nt + 1):
    u[j][0] = left_bc
    u[j][nx] = right_bc
# First time step j = 1 using initial velocity condition:
# u(i, 1) = (1 - λ²) u(i, 0) + 0.5 λ² [u(i+1, 0) + u(i-1, 0)] + k * g(x_i)
for i in range(1, nx):
    xi = i * h
    u[1][i] = (1.0 - lam2) * u[0][i] + 0.5 * lam2 * (u[0][i + 1] + u[0][i - 1]) + k * g_vel(xi)
# Subsequent time steps j >= 1
for j in range(1, nt):
    for i in range(1, nx):
        u[j + 1][i] = 2.0 * (1.0 - lam2) * u[j][i] + lam2 * (u[j][i + 1] + u[j][i - 1]) - u[j - 1][i]
```

```
print("\n--- Wave Displacement Table u(x, t) ---")
header = f"{'Time':<10}" + "".join(f"{f'x={i*h:.2f}':<10}" for i in range(nx + 1))
print(header)
print("-" * len(header))
for j in range(nt + 1):
    row_str = f"{j * k:<10.4f}" + "".join(f"{u[j][i]:<10.4f}" for i in range(nx + 1))
    print(row_str)
print("-" * len(header))
print(f"Wave simulation completed up to t = {total_time:.4f}.")
```

OUTPUT

Enter length of string (L): 2
Enter wave speed (c): 1
Enter number of spatial divisions (nx): 2
Enter number of time steps (nt): 4
Enter total time duration (T): 1
Enter initial displacement f(x) for u(x, 0) (press Enter for sin(pi*x/L)): x*(2-x)
Enter initial velocity g(x) = ∂u/∂t(x, 0) (press Enter for 0): 0
Enter left boundary condition at x = 0: 0
Enter right boundary condition at x = L: 0

20.FINITE DIFF WITH HYPERBOLIC(METHOD2)

```
import numpy as np
import math
L = float(input("Enter the length of the string (L): "))
c = float(input("Enter the wave speed (c): "))
M = int(input("Enter the number of spatial intervals (M): "))
T = float(input("Enter the final time (T): "))
N = int(input("Enter the number of time intervals (N): "))
if L <= 0 or c <= 0 or M < 2 or T <= 0 or N < 1:
    print("Invalid input.")
    raise SystemExit
h = L / M
k = T / N
r = c * k / h
print("\nSpace step h =", h)
print("Time step k =", k)
print("Courant number r =", r)
```

```
if r > 1:
    print("Stability condition violated")
    raise SystemExit
else:
    print("Stability condition satisfied")
x = np.linspace(0, L, M + 1)
def f(x):
    return 0.01 * np.sin(np.pi * x / L)
def g(x):
    return 0.0
U = np.zeros((N + 1, M + 1))
# Initial condition
for i in range(M + 1):
    U[0, i] = f(x[i])
# First time level
for i in range(1, M):
    U[1, i] = (
        f(x[i])
        + k * g(x[i])
        + (r ** 2 / 2)
        * (
            f(x[i + 1])
            - 2 * f(x[i])
            + f(x[i - 1])
        )
    )
# Boundary conditions
U[:, 0] = 0.0
U[:, M] = 0.0
# Explicit finite difference iteration
for n in range(1, N):
    for i in range(1, M):
        U[n + 1, i] = (
            2 * (1 - r ** 2) * U[n, i]
            + r ** 2 * (U[n, i - 1] + U[n, i + 1])
            - U[n - 1, i]
        )
    U[n + 1, 0] = 0.0
    U[n + 1, M] = 0.0
```

```
mid = M // 2
print("\nInitial displacement:")
print(f"u({x[mid]:.4f}, 0) = {U[0, mid]:.6f}")
print("\nFirst time-level displacement:")
print(f"u({x[mid]:.4f}, {k:.6f}) = {U[1, mid]:.6f}")
print("\nFinal displacement at time T =", T)
print("x\t\tU(x,T)")
step = max(1, M // 10)
for i in range(0, M + 1, step):
    print(f"{x[i]:.4f}\t\t{U[N, i]:.6f}")
if M % step != 0:
    print(f"{x[M]:.4f}\t\t{U[N, M]:.6f}")
print("\nNumerical solution completed.")
```

OUTPUT

Enter the length of the string (L): 2

Enter the wave speed (c): 1

Enter the number of spatial intervals (M): 2

Enter the final time (T): 1

Enter the number of time intervals (N): 4